ClaimIQ

Deployment Plan & Production Architecture

Malaysian TPA Claims Intelligence Platform

Version 2.0 | May 2026 | UMHackathon 2026

Platform	FastAPI + Vanilla JS
Cloud Host	Vercel (Serverless)
CI/CD	GitHub Actions -> Vercel
AI Services	Z.AI GLM-4-plus + Google Gemini
Live URL	https://tpa-blue.vercel.app

Table of Contents

Table of Contents 2
1. System Overview 4
  1.1 Technical Stack 4
  1.2 Core Processing Pipeline (8 Steps) 4
2. Production Architecture 6
  2.1 Vercel Serverless Deployment 6
    Entry Point 6
    Key Serverless Constraints & Solutions 6
  2.2 URL Structure 6
  2.3 Database Architecture 7
  2.4 AI Service Integration 7
3. CI/CD Pipeline 8
  3.1 Pipeline Overview 8
  3.2 GitHub Actions Workflow 8
    Trigger Conditions 8
    Job Structure 9
  3.3 Deployment Trigger History 9
4. Credential & Secret Management 10
  4.1 Principles 10
  4.2 Environment Variable Registry 10
  4.3 GitHub Secrets Setup 11
  4.4 CORS Security 11
5. Scalability & Production Upgrade Path 12
  5.1 Current Architecture Limitations 12
  5.2 Production-Ready Upgrade Path 12
  5.3 Vercel Auto-Scaling 12
6. Security Architecture 13
  6.1 Authentication & Authorization 13
  6.2 Input Validation & Injection Prevention 13
  6.3 PDPA Compliance (Personal Data Protection Act 2010) 13
  6.4 API Rate Limiting & Abuse Prevention 13
7. Local Development Setup 14
  7.1 Prerequisites 14
  7.2 Setup Steps 14
  7.3 Environment Variables for Local Dev 14

7.4 Seeding Demo Data 14
8. Key Bug Fixes & Hardening Implemented 15
9. Deployment Commit History 16
9.1 Repository 16

1. System Overview

ClaimIQ is a web-based TPA (Third-Party Administrator) claims intelligence platform purpose-built for Malaysian healthcare workflows. It automates the full claims lifecycle from intake and document parsing to AI-powered adjudication, fraud detection, eligibility verification, and bilingual Explanation of Benefits (EOB) generation.

1.1 Technical Stack

Layer	Technology	Purpose
Backend API	FastAPI (Python 3.11)	REST API, business logic, pipeline orchestration
Frontend	Vanilla HTML/CSS/JS	Single-page UI, served as static files by FastAPI
Database	SQLite (/tmp on Vercel)	Claims, decisions, audit logs, fraud scores, tokens
LLM Adjudication	Z.AI GLM-4-plus (OpenAI-compatible)	Claim extraction, coding, adjudication, fraud, advisory
Document AI	Google Gemini 2.5 Flash	Invoice/lab/X-ray parsing (image + PDF)
Policy RAG	FAISS + sentence-transformers	Semantic policy clause retrieval for adjudication
Authentication	PyJWT (HS256)	Role-based access: ADMIN, ADJUDICATOR, CLINIC_USER
PDF Generation	ReportLab	Bilingual EN/BM Explanation of Benefits PDF
Hosting	Vercel Serverless	Auto-scaling Python WSGI functions
CI/CD	GitHub Actions	Automated test + deploy pipeline on every push

1.2 Core Processing Pipeline (8 Steps)

Every claim submitted to ClaimIQ flows through a deterministic, auditable 8-step pipeline:

1. Claim Intake & Record Creation -- claim text, patient metadata, and base64-encoded evidence attachments are persisted to SQLite.
2. Structured Extraction (GLM) -- GLM-4-plus extracts structured fields (diagnosis, ICD-10, amounts, dates) from free-text claim description.

3. Evidence Parsing (Gemini) -- Gemini parses attached invoice images, lab report images/PDFs, and X-rays into structured JSON. Extracts patient name, IC number, line items, lab values, and flags.
4. Deterministic Cross-Reference -- ValidationEngine and CrossReferenceEngine compare claim vs. evidence for identity mismatches, date discrepancies, amount contradictions, and elevated lab flags vs. doctor notes.
5. Claim Scrubbing -- duplicate detection (CARC 18), missing fields, timely filing checks (14-day rule).
6. Eligibility Verification -- IC-based member lookup, benefit limit checks, co-pay calculation (5% BNM standard), exclusion screening.
7. Disposition Engine (Hard Gates) -- deterministic policy rules (ID_MATCH_001, TF_LIMIT_001, ELIG_001, PARSE_001) can finalize a claim immediately without LLM.
8. AI Adjudication + Fraud + Advisory -- GLM produces clinical reasoning with evidence citations; fraud scoring (graph signals, behavioral analytics); GP advisory in plain language.

2. Production Architecture

2.1 Vercel Serverless Deployment

ClaimIQ runs as a serverless Python application on Vercel. Vercel detects the FastAPI app exported from main.py and wraps it in a WSGI handler, distributing requests across auto-scaled function instances in Vercel's global edge network.

Entry Point

`main.py` -- exports `app` from `execution/api_server.py`. Vercel auto-discovers this and configures its Python runtime accordingly.

Key Serverless Constraints & Solutions

Constraint	Impact	Solution Implemented
Read-only filesystem	Cannot write SQLite to default paths	DB_PATH forced to /tmp/claimiq.db (writable on Vercel)
No persistent memory	Threads don't share state across invocations	All processing runs synchronously within the request handler
4.5 MB payload limit	Large base64 images exceed limit	Images compressed to <800 KB before inline base64 encoding
Cold starts	First request after idle is slow	Lazy-loading for heavy dependencies (FAISS, PIL, Gemini SDK)
bcrypt unavailable	C extension fails on Vercel Python runtime	Replaced with hashlib PBKDF2-SHA256 (pure Python)

2.2 URL Structure

Environment	URL	Notes
Production (Stable)	`https://tpa-blue.vercel.app`	Primary production domain -- always points to latest deployed build
Production (Per-deploy)	`https://tpa-[hash].vercel.app`	Immutable URL for each individual deployment -- useful for rollbacks
Local Dev	`http://127.0.0.1:8000`	Run: uvicorn main:app --reload

API Docs	`/docs` | `/redoc`	FastAPI auto-generated Swagger/ReDoc (disabled in production)
Health Check	`/api/health`	Returns runtime status, DB connectivity, env var presence
Debug Endpoint	`/api/debug/env`	Live-tests GLM + Gemini APIs, reports all env var states (admin only)

2.3 Database Architecture

SQLite is used as the persistence layer. On Vercel, the database file lives in /tmp which is ephemeral per-function-instance (reset on cold starts). This is acceptable for a hackathon/demo deployment; a production upgrade path is outlined in Section 5.

Core tables:

- claims -- lifecycle, status, patient data, financial fields, processing metadata
- decisions -- full adjudication payloads (JSON), versioned per processing run
- fraud_scores -- fraud risk scores, flags, graph signals
- audit_logs -- PDPA-compliant audit trail with data classification, retention periods
- action_notes -- appeals, RAI responses, review actions
- advisories -- GP advisory outputs per claim
- users -- role-based accounts (ADMIN / ADJUDICATOR / CLINIC_USER)
- token_usage -- GLM token consumption per call, accumulated for cost dashboard

2.4 AI Service Integration

Service	Provider	Model	Usage
LLM (Adjudication)	Z.AI / Ilmu AI	GLM-4-plus	Extraction, coding, adjudication, fraud, advisory
Document Vision	Google Gemini	gemini-2.5-flash	Invoice, lab report, X-ray parsing (image + PDF)
Policy RAG	Local (FAISS)	all-MiniLM-L6-v2	Semantic policy clause retrieval

Both AI APIs are called via HTTPS with API keys stored as Vercel environment secrets -- never in source code. Fallback behaviour: if GLM is unavailable in production (APP_ENV=production), the system raises GLMServiceUnavailable and routes the claim to UNDER_REVIEW for manual handling.

3. CI/CD Pipeline

ClaimIQ uses GitHub Actions for fully automated continuous integration and deployment. Every push to the main branch triggers the pipeline. Deployments to Vercel only proceed if all tests pass.

3.1 Pipeline Overview

#	Stage	What Happens	Gate / Outcome
1	**Code Push**	Developer pushes commits to GitHub (branch: main or master)	Triggers CI/CD workflow automatically
2	**Checkout + Setup**	GitHub Actions checks out repo; installs Python 3.11 via setup-python@v6	Environment ready for test run
3	**Dependency Install**	pip install -r requirements.txt + pytest	Fails build if any package is missing
4	**Unit Tests**	pytest -q tests/ -- runs scrubber, EOB, fraud, disposition, validation engine tests	GATE: Deploy blocked if any test fails
5	**Vercel Auth Pull**	vercel pull --yes -- environment=production -- syncs env vars and project config from Vercel cloud	Uses VERCEL_TOKEN, VERCEL_ORG_ID, VERCEL_PROJECT_ID secrets
6	**Build Artifacts**	vercel build --prod -- compiles the project to Vercel's output format (.vercel/output/)	Fails if build config is invalid
7	**Deploy to Production**	vercel deploy --prebuilt --prod -- pushes the pre-built artifacts directly to Vercel's CDN	Live at tpa-blue.vercel.app within ~30 seconds

3.2 GitHub Actions Workflow

File location: .github/workflows/ci.yml

Trigger Conditions

- on: push -> branches: [main, master] -- fires on every direct push
- on: pull_request -- runs tests only (no deploy) on PRs for safety review

Job Structure

The workflow is split into two sequential jobs:

Job	Runs On	Condition
test	ubuntu-latest	Always (push + PR)
deploy-vercel	ubuntu-latest (needs: test)	Only on push to main/master -- never on PRs

The needs: test dependency ensures deploy-vercel cannot start until the test job completes successfully. This is the primary safety gate preventing broken code from reaching production.

3.3 Deployment Trigger History

During development, several force-trigger commits were used to initiate redeployment when environment variables changed on Vercel (env changes alone do not trigger GitHub Actions):

- Commit message pattern: "trigger: force CI/CD redeployment for Vercel" -- pushes an empty/metadata commit to fire the workflow
- This approach ensures Vercel picks up new secret values (e.g., GLM API keys, DB_PATH) without requiring a code change

4. Credential & Secret Management

4.1 Principles

- Zero secrets in source code -- all credentials stored in Vercel Environment Variables (encrypted at rest)
- .env files are excluded from version control via .gitignore (.tmp/, .env, *.db)
- .env.example is committed to guide local setup without exposing real values
- API keys are redacted from all server logs before persistence
- JWT tokens use HS256 with server-side secret; token expiry enforced

4.2 Environment Variable Registry

Variable	Required	Secret	Purpose
ZAI_API_KEY / ILMU_API_KEY	Yes	Yes	GLM-4-plus LLM API authentication
ZAI_BASE_URL / ILMU_BASE_URL	Yes	No	LLM API endpoint URL
ZAI_MODEL / ILMU_MODEL	No	No	Model name override (default: glm-4-plus)
gemini / GEMINI_API_KEY	Yes	Yes	Google Gemini document vision API key
GEMINI_MODEL	No	No	Override Gemini model (default: gemini-2.5-flash)
APP_ENV	Yes	No	Set to 'production' on Vercel -- enables prod error handling, disables mock fallback
DB_PATH	Yes	No	Database file path -- must be /tmp/claimiq.db on Vercel
API_BEARER_TOKEN	No	Yes	Optional: bearer token for additional API endpoint protection
ALLOWED_ORIGINS	No	No	CORS whitelist (comma-separated). Defaults to * if unset
DEMO_ADMIN_PASSWORD	No	Yes	Required in production to seed demo admin user -- prevents default credentials
VERCEL_TOKEN	CI only	Yes	GitHub Actions secret -- authorizes vercel CLI to deploy
VERCEL_ORG_ID	CI only	Yes	GitHub Actions secret -- Vercel organisation identifier

VERCEL_PROJECT_ID	CI only	Yes	GitHub Actions secret -- Vercel project identifier (prj_FrpuFt...)

4.3 GitHub Secrets Setup

Three secrets are configured in the GitHub repository Settings -> Secrets -> Actions:

- VERCEL_TOKEN -- generated from Vercel account settings (Account -> Tokens)
- VERCEL_ORG_ID -- found in Vercel project settings or .vercel/project.json
- VERCEL_PROJECT_ID -- found in .vercel/project.json after vercel link

These secrets are injected as environment variables into the deploy-vercel GitHub Actions job and are never printed to logs.

4.4 CORS Security

CORS is configured from the ALLOWED_ORIGINS environment variable to prevent unauthorized cross-origin access:

- Development: defaults to * (all origins) when ALLOWED_ORIGINS is not set
- Production: set ALLOWED_ORIGINS=https://tpa-blue.vercel.app to lock down to the production domain only
- Multiple origins supported as a comma-separated list

5. Scalability & Production Upgrade Path

5.1 Current Architecture Limitations

The current deployment uses SQLite on Vercel's ephemeral /tmp filesystem, which means:

- Data does not persist across cold starts or new function instances
- Concurrent writes under high load can cause SQLite locking
- This is acceptable for hackathon/demo use -- all core logic is production-ready

5.2 Production-Ready Upgrade Path

Component	Current (Demo)	Production Upgrade
Database	SQLite (/tmp)	PostgreSQL on Supabase, Neon, or AWS RDS -- change DB_PATH to connection string
Hosting	Vercel Serverless	Vercel Pro (higher limits) or AWS ECS Fargate for persistent containers
Auth	JWT HS256 (local)	Auth0 / Okta / AWS Cognito for enterprise SSO, MFA, audit compliance
File Storage	In-memory base64	AWS S3 / Azure Blob -- store evidence files with signed URLs
Policy Index	Local FAISS (.tmp/)	Pinecone or Weaviate vector DB for persistent semantic search
Monitoring	Server logs only	Datadog / Sentry / CloudWatch -- error alerting, latency dashboards, cost tracking
Secrets	Vercel env vars	AWS Secrets Manager / HashiCorp Vault for rotation and auditing

5.3 Vercel Auto-Scaling

Vercel's serverless functions scale automatically from 0 to N concurrent instances with no configuration required:

- Each API request is isolated in its own function invocation
- No server management, OS patching, or capacity planning needed
- Global CDN edge caches static frontend assets (HTML, CSS, JS) worldwide
- Python function cold start: ~2-4 seconds; subsequent requests: <500 ms

6. Security Architecture

6.1 Authentication & Authorization

- JWT tokens issued on login, expire after configurable TTL
- Role-based access control: ADMIN (full access), ADJUDICATOR (process/review), CLINIC_USER (submit/view own claims)
- Tenant isolation: multi-clinic deployments filter all DB queries by clinic_id
- Passwords hashed with PBKDF2-SHA256 (hashlib, 100,000 iterations) -- bcrypt was replaced due to Vercel's Python runtime C-extension limitations

6.2 Input Validation & Injection Prevention

- SQLite queries use parameterized statements throughout -- no string interpolation
- Column injection guard: update_claim() validates column names against a frozenset whitelist before executing any UPDATE
- File upload: base64 is decoded and routed by MIME magic bytes (PDF: %PDF, images: PIL validation) -- no user-controlled file extensions trusted
- Gemini prompt injection: extracted document fields are sanitized before being injected into LLM prompts

6.3 PDPA Compliance (Personal Data Protection Act 2010)

- Audit log includes: data_classification, retention_period, data_minimization_applied, consent_basis, cross_border_transfer fields
- Patient IC numbers are masked in logs and API responses where possible
- Claim export endpoint (GET /api/claims/{id}/export) supports data portability
- Demo user seeding is gated in production -- requires DEMO_ADMIN_PASSWORD env var to prevent default credentials

6.4 API Rate Limiting & Abuse Prevention

- Optional API_BEARER_TOKEN provides an additional layer of endpoint protection
- Processing lock mechanism prevents concurrent reprocessing of the same claim
- GLM API key redacted from all error logs before persistence: raw_err.replace(api_key, '***REDACTED***')

7. Local Development Setup

7.1 Prerequisites

- Python 3.11+
- Node.js 22+ (for docx skill scripts only)
- Git

7.2 Setup Steps

9. Clone the repository: git clone https://github.com/anas-adil/CLAIMIQ-V2.git
10. Install Python dependencies: pip install -r requirements.txt
11. Copy .env.example to .env and fill in API keys
12. Run the development server: uvicorn main:app --reload --port 8000
13. Access UI at http://127.0.0.1:8000
14. Run tests: pytest -q tests/

7.3 Environment Variables for Local Dev

Variable	Local Value
APP_ENV	dev (enables mock fallback when LLM unavailable)
DB_PATH	.tmp/claimiq.db (relative path, auto-created)
ZAI_API_KEY	Your Z.AI or llmu AI key
gemini	Your Google Gemini API key
ALLOWED_ORIGINS	(leave unset -- defaults to * for local dev)

7.4 Seeding Demo Data

To populate the system with sample claims for demonstration:

- POST /api/demo/seed -- creates demo users and sample claims
- POST /api/demo/generate -- generates additional synthetic claims
- In production: requires DEMO_ADMIN_PASSWORD to be set

8. Key Bug Fixes & Hardening Implemented

During the development and deployment phase, the following production-critical issues were identified and resolved:

#	Issue	Root Cause	Fix
1	**bcrypt 500 error on Vercel**	C extension not available in Vercel Python runtime	Replaced with hashlib PBKDF2-SHA256
2	**SQLite read-only crash**	Default DB path was in read-only Vercel filesystem	Force DB_PATH=/tmp/claimiq.db
3	**Background thread freeze**	asyncio.create_task blocked in Vercel WSGI context	Switched to synchronous processing
4	**RM 0.00 in adjudication**	GLM coding response overwrote submitted total with 0	Re-assert amount from claim record after LLM merge
5	**PDF lab reports not parsed**	PIL.Image.open() silently fails on PDFs	Detect %PDF magic bytes, route to Gemini as application/pdf
6	**Identity mismatch not caught**	Gemini prompts lacked IC number fields; normalize_name stripped spaces	Added IC fields to prompts; fixed regex to [^A-Z]
7	**Copay charged 100% of bill**	Math bug: min(total, max(5%, min(500, total))) = total	Fixed to clamp(5%, RM5, RM500) then min(copay, total)
8	**No adjudication reasoning shown**	Empty string in DB not caught; JS falsy check showed fallback	Changed merge check from is None to not .get(key)
9	**SQL column injection risk**	update_claim() accepted arbitrary column names	Whitelist frozenset guard on all column updates
10	**Connection leaks**	DB connections only closed on happy path	Wrapped all DB operations in try/finally blocks
11	**Malaysian name false mismatches**	Token similarity used max denominator -- short names scored 0.5	Changed to min denominator -- short name subset scores 1.0
12	**Duplicate claim soft-fail**	Exact duplicates only produced warnings, not hard deny	Upgraded to errors with CARC 18 -- hard FAIL
13	**IndentationError on deploy**	conn.close() not indented inside finally block	Fixed indentation -- server restored immediately

9. Deployment Commit History

All changes are version-controlled on GitHub (anas-adil/CLAIMIQ-V2, branch: main). Key commits:

Commit	Description
1b426b6	fix: add IC extraction to Gemini prompts and fix _normalize_name space-stripping bug
9967803	fix: correct indentation of conn.close() inside finally block in rai-response endpoint
9ecb01c	fix: comprehensive production hardening -- 13 bugs fixed across 12 files
d2f7923	fix: mock adjudication extraction, debug endpoint, amount SQLite safety
7e6d48f	refactor: replace hardcoded lab-check list with fully dynamic comparison
c0f9c69	fix: PDF parsing, amount override, and elevated-lab contradiction detection
b28f46c	feat: strengthen adjudication reasoning -- compare Gemini evidence vs claim
af9ae3f	fix: run claim processing synchronously on Vercel to prevent background threads
9232a2a	fix: enforce /tmp path for sqlite DB on Vercel to bypass read-only filesystem crash
534b778	fix: Replace bcrypt with hashlib to prevent 500 Server Error on Vercel Python runtime

9.1 Repository

GitHub: https://github.com/anas-adil/CLAIMIQ-V2

Branch: main | Auto-deployed to Vercel on every push